Database Normalization

ALS SDO San Juan | First, Second, and Third Normal Forms

This report evaluates the database structure defined by the project migrations. It is a schema-level review, not a live database or data-quality audit. The current design appears to satisfy 1NF and 2NF and is broadly in 3NF, subject to the business-rule checks noted below.

Assessment at a glance

Normal form	Requirement	Assessment of this database
1NF	One value per field; repeating sets represented as separate rows.	Meets: profile attributes are individual fields; documents, enrollments, progress records, and game attempts have separate rows.
2NF	1NF, and each non-key attribute depends on the whole candidate key.	Meets based on the declared keys. Progress is unique per learner and module; game attempts are unique per learner and attempt UUID.
3NF	2NF, and non-key attributes do not depend on other non-key attributes.	Broadly meets: related facts are split across entity tables and linked by foreign keys. Review location, prior-study history, and teacher assignment rules.

How the forms apply

A hypothetical single intake sheet could put a learner, profile, center, enrollments, document uploads, module progress, and game attempts into one record. Multiple documents or attempts would then need repeated columns or lists, and learner details would be copied across rows. That is an illustration of what the design avoids; it is not a table currently defined in the project.

1NF | First Normal Form

A relation is in 1NF when each field contains one value and each row represents one record. The project stores repeatable activities as rows instead of fields such as document_1, document_2, or module_1. Examples include:

Table	One row represents
user_profiles	One person profile linked to one user account.
enrollments	One learner enrollment for a program and year.
learner_documents	One uploaded document and its review details.
module_downloads_and_progress	One learner's progress record for one module.
module_game_attempts	One game attempt by one learner.

Result: the application tables use atomic fields and separate rows for repeatable records, so the schema meets 1NF based on the migration definitions.

2NF | Second Normal Form

2NF adds a whole-key dependency rule: if a row is identified by more than one field, its non-key values must depend on the complete key. A learner's name depends on the learner, a module title depends on the module, and reading progress depends on the learner-module pairing.

Fact	Functional dependency	Where it belongs in the current schema
Learner account details	user_id -> username, email, role, status	users
Learner demographic details	user_id -> profile fields	user_profiles (user_id is unique)
Module details	module_id -> title, strand_id, level, language, and file metadata	modules
Reading progress	(user_id, module_id) -> progress fields	module_downloads_and_progress; pair is unique
Game attempt details	(user_id, attempt_uuid) -> attempt fields	module_game_attempts; pair is unique

The schema uses a single-column surrogate primary key in its tables. It also declares logical unique keys for progress and game attempts. No partial dependency is evident in the application tables from the available migrations.

2NF decomposition example

If learner name, module title, and last page read were stored together in a learner-module table, learner name would depend only on user_id and module title only on module_id. The current design keeps account/profile facts in user tables, module facts in modules, and the paired reading state in module_downloads_and_progress. This removes those partial dependencies.

3NF | Third Normal Form

3NF removes transitive dependencies, where a non-key value is determined by another non-key value instead of directly by the row key. The project already separates several such subjects:

Relationship / dependency	Normalization in the current schema
strand_id -> strand code, name, description, display settings	Stored once in learning_strands; modules reference it through strand_id.
module_id -> module metadata and strand_id	Stored in modules; progress and attempts reference module_id rather than copying module details.
user_id -> account fields; user_id -> profile fields	Account data is in users; demographic data is in the one-to-one user_profiles table.
user_id + module_id -> progress fields	Progress remains in its own learner-module table.

This separation reduces update anomalies: changing a strand name, module title, or learner profile does not require updating multiple progress or attempt rows.

Current normalized table structure

Subject	Current table(s)	Key relationship
Accounts and learner profiles	users; user_profiles	One profile per user; profile.user_id references users.id.
Centers and enrollment	community_learning_centers; enrollments	Enrollment references a learner, center, and optional teacher.
Uploaded records	learner_documents	Each document references a learner and may reference an enrollment and reviewer.
Learning content	learning_strands; modules	Each module references a learning strand and its uploader.
Learning activity	module_downloads_and_progress; module_game_attempts	Both reference users and modules; attempts are stored as individual events.
Portal content	hero_banners	Banner content is stored independently of learner records.

Potential 3NF refinements to confirm

Area	Why review it	Possible normalized design
Address fields	If barangay determines city and province, storing all three on a profile creates a transitive dependency: profile_id -> barangay -> city/province.	Add location reference tables (for example, barangays linked to a city) and store a barangay foreign key. If city/province are fixed constants for this system, document that assumption.
Prior ALS history	The profile has fields for one prior ALS record. If a learner can have multiple prior programs or attendance periods, the profile cannot represent each record independently.	Use a learner_education_history table with one row per prior program/period.
Teacher assignment	A center has an assigned teacher and an enrollment also has an assigned teacher. If an enrollment always inherits the center's teacher, the same fact may be stored twice.	Keep one authoritative assignment, or retain enrollment-level assignment when it is an intentional override or historical snapshot.

These are conditional findings: confirm the actual business rules before changing the schema. Enum fields such as role, status, program track, and delivery mode are atomic values and do not by themselves violate normalization. Lookup tables are optional when their values are simple and stable.

Conclusion

The migration-defined ALS application schema is already organized through 1NF and 2NF and is broadly in 3NF. The main model separates users, profiles, centers, enrollments, documents, strands, modules, progress, and attempts. A stricter 3NF claim depends on the location, repeatable education-history, and teacher-assignment rules being clarified. This report does not change the database.

Laravel also defines support tables for sessions, cache, queued jobs, failed jobs, password resets, and API tokens. These are framework storage structures and are separate from the ALS learner-domain normalization assessment.